Package ‘USFarmSafetyNetLab’

August 3, 2025

Type Package

Title US Farm Safety Net Lab

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<https://orcid.org/0000-0001-5984-1072>)

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Contributor -

Reviewer -

Creator Francis Tsiboe

Description This repository centralizes research outputs, analytical tools, and resources for exploring and evaluating the United States agricultural safety net programs. It supports analysis of key programs including the Federal Crop Insurance Program (FCIP), the Noninsured Crop Disaster Assistance Program (NAP), Price Loss Coverage (PLC), Agricultural Risk Coverage (ARC), and various ad-hoc disaster assistance programs.

License GPL-3 + file LICENSE

URL https://github.com/you/USFarmSafetyNetLab

BugReports https://github.com/you/USFarmSafetyNetLab/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

VignetteBuilder knitr

Depends R (>= 4.1.0)

Imports rfcip, data.table, dplyr, doBy, piggyback, plm, purrr, readr, rvest, sp, spdep, stringr, terra, classInt, janitor, tidyr, tigris, xml2, moments, stats, sf, utils, urbnmapr, ggplot2, gridExtra, gtable

Remotes github::dylan-turner25/rfcip, github::UrbanInstitute/urbnmapr

Suggests knitr, rmarkdown, testthat (>= 3.0.0)

LazyData true

Cite-us If you find it useful, please consider staring the repository and citing the following studies
- Tsiboe, F. and Turner, D. (2025). ``Incorporating buy-up price loss coverage into the United States farm safety net." Applied Economic Perspectives and Policy.
- Tsiboe, F., et al. (2025). ``Risk reduction impacts of crop insurance in the United States."

Applied Economic Perspectives and Policy.
- Gaku, S. and Tsiboe, F. (2024). Evaluation of alternative farm safety net program combination strategies. Agricultural Finance Review.

Contents

analyze_and_summarize_time_series . . . 3
calculate_mode . . . 4
calculate_revenue_moments . . . 5
census_state_beginning_farmer_and_rancher_data . . . 6
clean_data . . . 6
contiguous_county . . . 7
downloaded_nass_large_datasets . . . 7
download_rma_web_data_files . . . 8
ers_theme . . . 9
estimate_fcip_instruments . . . 9
estimate_fcip_unloaded_rate . . . 10
FCIP_FORCE_CHARACTER_KEYS . . . 12
FCIP_FORCE_NUMERIC_KEYS . . . 12
FCIP_INSURANCE_ELECTION . . . 13
FCIP_INSURANCE_ELECTION_RCODED . . . 14
FCIP_INSURANCE_POOL . . . 15
fixed_effect_model_data_prep . . . 16
generate_toy_farmer_data . . . 17
get_data_release . . . 18
get_file_info . . . 19
get_ice_data . . . 20
get_marketing_year_avg_price . . . 21
get_nass_census_data . . . 23
get_nass_historical_track_record_crop . . . 24
get_nass_production_data . . . 25
get_state_rental_rates . . . 26
get_yu2018_instrument . . . 27
harmonize_crop_type_codes . . . 28
harmonize_election_codes . . . 29
index_for_price_recived . . . 30
locate_download_link . . . 30
marketing_year_avg_price . . . 31
panel_based_spatial_smoothing_estimator . . . 31
plot_liability_and_acres . . . 32
policytime . . . 34
prep_fcip_data . . . 35
prep_nass_data . . . 36
process_nass_dataset . . . 37
rma_type_recodes . . . 39
smooth_county_estimates . . . 39
standardize_fcip_column_names . . . 40
state_rental_rates . . . 41

Index **42**

`analyze_and_summarize_time_series`
Analyze and summarize time-series data by disaggregate and outcome using data.table

Description

This function computes comprehensive summary statistics, trend diagnostics, and both magnitude-sorted and chronological notable changes for each combination of `disaggregate` level and `outcome`.

Usage

```
analyze_and_summarize_time_series(dt, threshold = 10)
```

Arguments

`dt`	A data.table or data.frame containing the columns: • `commodity_year` (integer) • `disaggregate` (character or factor) • `outcome` (character or factor) • `value` (numeric)
`threshold`	Numeric. Percent threshold (e.g., 10 for 10%) used to flag `notable_growth` (>= +threshold) and `notable_decline` (<= -threshold). Defaults to 10.

Value

A named list of outcomes. Each element is a named list of one-row data.frames (one per disaggregate level) containing the following summary columns:

- **mean**: Arithmetic mean of the `value` series.
- **sd**: Standard deviation of the `value` series.
- **median**: Median of the `value` series.
- **iqr**: Interquartile range (75th-25th percentile) of the `value` series.
- **cv**: Coefficient of variation (sd divided by mean).
- **n_obs**: Number of non-missing observations.
- **pct_missing**: Percentage of years with missing `value`.
- **initial_year_value**: Value in the first `commodity_year` period.
- **final_year_value**: Value in the last `commodity_year` period.
- **year_with_max_value**: Year when `value` is maximized.
- **max_value**: Maximum `value` observed.
- **year_with_min_value**: Year when `value` is minimized.
- **min_value**: Minimum `value` observed.
- **cumulative_change**: Absolute change (final minus initial value).
- **mean_annual_growth_rate**: Compound annual growth rate (CAGR) in percent.
- **growth_initial_to_final**: Total percent growth from initial to final value.

- **first_diff_sd**: Standard deviation of year-to-year percent changes.
- **max_drawdown_pct**: Greatest percent drop from a peak to subsequent trough.
- **trend_slope**: Slope of linear model `value ~ commodity_year`.
- **trend_p_value**: P-value for slope significance in the trend model.
- **trend_r_squared**: R-squared of the trend model.
- **time_to_peak**: Years from start until the peak year.
- **time_to_trough**: Years from start until the trough year.
- **notable_growth**: Magnitude-sorted list of year: +% changes greater than or equal to threshold.
- **notable_growth_chron**: Chronologically-sorted list of year: +% changes greater than or equal to threshold.
- **notable_decline**: Magnitude-sorted list of year: -% changes less than or equal to -threshold.
- **notable_decline_chron**: Chronologically-sorted list of year: -% changes less than or equal to -threshold.
- **any_missing_years**: Logical flag for missing years in the series.
- **monotonic**: Logical flag if the series strictly increases or decreases.
- **rank**: Rank of the disaggregate within its outcome by descending CAGR.
- **rank_among_outcomes**: Global CAGR rank across all outcome/disaggregate groups.

`calculate_mode` *Calculate the Statistical Mode*

Description

Returns the element that occurs most frequently in a vector.

Usage

```
calculate_mode(x, na.rm = TRUE)
```

Arguments

`x`	A vector of any atomic type (numeric, character, factor,).
`na.rm`	Logical; should missing values be ignored? Defaults to `TRUE`. If `FALSE` and `x` contains any `NA`s, the function returns `NA`.

Details

Internally the function:

1. Optionally removes `NA`s (`na.rm = TRUE`).
2. Builds a lookup table of unique values via `unique(x)`.
3. Counts the frequency of each unique value with `tabulate(match(x, ux))`.
4. Returns the value with the maximum count.

Because it relies on base R functions, the implementation is vectorised and generally fast for typical data-frame column sizes.

Value

A single value giving the modal element of `x`. If two or more values are tied for the highest frequency, the first one encountered in `x` is returned.

---

calculate_revenue_moments

Calculate key distributional moments for a numeric vector of revenues

---

Description

Calculate key distributional moments for a numeric vector of revenues

Usage

```
calculate_revenue_moments(x)
```

Arguments

`x`	Numeric vector of revenues (e.g. per-draw revenues)

Value

A one-row data.frame with:

- mu: mean
- md: median
- sd: standard deviation
- cv: coefficient of variation (sd / mu)
- vr: variance
- sk: skewness
- ku: kurtosis
- lapv: mean squared loss deviations below the mean
- lrpv: lapv / probability of loss
- nlapv: lapv normalized by mu
- nlrpv: lrpv normalized by mu
- lres2: same as lapv (for clarity)
- cdf: empirical probability of loss (P(X < mu))

census_state_beginning_farmer_and_rancher_data *Simulator Helper Datasets*

Description

A combined dataset for census_state_beginning_farmer_and_rancher_data

Usage

```
data(census_state_beginning_farmer_and_rancher_data)
```

Format

A data frame with 255 rows and 16 columns covering Inf–Inf.

Source

USDA NASS Quick Stats

clean_data *Apply standardized data cleaning opperations*

Description

Apply standardized data cleaning opperations

Usage

```
clean_data(df)
```

Arguments

df	a data frame to clean

Value

a cleaned data frame

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run: clean_data(df)
```

contiguous_county *contiguous_county*

Description

A combined dataset for contiguous_county

Usage

```
data(contiguous_county)
```

Format

A data frame with 24307 rows and 12 columns covering Inf–Inf.

Source

USDA-RMA, Actuarial Data Master - A0123

downloaded_nass_large_datasets *Download and cache USDA NASS Quick Stats large dataset files*

Description

`downloaded_nass_large_datasets()` retrieves a Quick Stats file from the USDA National Agricultural Statistics Service (NASS) https://www.nass.usda.gov/datasets/ page and saves it locally. If the file is already present in the target directory, it is not re-downloaded.

Usage

```
downloaded_nass_large_datasets(
  large_datasets,
  dir_dest = "./data-raw/fastscratch/nass/"
)
```

Arguments

`large_datasets`	`character list` The base name of the Quick Stats file to download. For example, use `"crops"` to fetch `qs.crops_YYYYMMDD.txt.gz` or include `"census2022"` (e.g. `"census2022"`) to fetch the gzipped 2022 census version (`qs.census2022.txt.gz`). any of: "census2002","census2007","census2012","census2017","census2022", "census2007zipcode","census2017zipcode", "animals_products","crops","demographics","economic
`dir_dest`	`character(1)` Path to a directory where downloaded files will be stored. Defaults to `"./data-raw/fastscratch/nass/"`.

Details

1. Prepends "qs." to the provided large_dataset. If large_dataset contains "census", appends ".txt.gz", otherwise NULL.
2. Ensures dir_dest exists (creates it if needed).
3. Scrapes the NASS datasets page (https://www.nass.usda.gov/datasets/) for links ending in .txt.gz.
4. Downloads the matching file into dir_dest if not already present.

Value

Invisibly returns the normalized file large_dataset (e.g. "qs.crops_YYYYMMDD.txt.gz" or "qs.censusYYYY.txt.gz") that was downloaded or already present.

Examples

```
## Not run:
# Download the 'crops' dataset if not already cached:
downloaded_nass_large_datasets(large_dataset = "crops")

# Download the 2022 census version:
downloaded_nass_large_datasets(large_dataset = "census2022",
dir_dest = "./data-raw/fastscratch/nass/")

## End(Not run)
```

---

download_rma_web_data_files

Download and Process RMA Web Data Files

Description

Fetches one or more years of USDA RMA Summary of Business data (state-county-crop & livestock participation series), unzips, reads the pipe-delimited text, applies the correct column names, and saves each year as an .rds under dest.

Usage

```
download_rma_web_data_files(
  years,
  file_name,
  dest = "./data-raw/data_release",
  url_rma_web_data_files = "https://pubfs-rma.fpac.usda.gov/pub/Web_Data_Files"
)
```

Arguments

years	Integer vector of crop or livestock years to fetch.
file_name	Character; one of "sobcov", "sobtpu", "lgm", "lrp", or "colsom". Determines both the subdirectory and the column schema.
dest	Character path where the final .rds files will go. Defaults to "./data-raw/data_release".

`url_rma_web_data_files`
: Base URL for USDA RMA web data. Defaults to the official RMA Web_Data_Files root.

Value

Invisibly returns `NULL`. Side effect: one `.rds` per year is written under `dest`, named `<file_name>_<year>.rds`.

Examples

```
## Not run:
# Get sobtpu data for 2018-2022
download_rma_web_data_files(2018:2022, "sobtpu")

## End(Not run)
```

---

ers_theme *ERS Theme*

Description

ERS Theme

Usage

```
ers_theme()
```

Source

coppied from https://github.com/USDA-REE-ERS/MTED-Theme on 08/01/2025

Examples

```
ggplot2::ggplot() + ers_theme()
```

---

estimate_fcip_instruments *Estimate FCIP Instrumental Variables (Unloaded Rates)*

Description

Uses historical FCIP rate data to build instrumented unloaded-rate variables following:

1. Tsiboe & Turner (2023), Econometric identification of crop insurance participation *Agricultural and Resource Economics Review*, 52(3):476-497. https://doi.org/10.1017/age.2023.13

Usage

```
estimate_fcip_instruments(year, statplan)
```

Arguments

`year` Integer. The target crop year for which to construct instruments.

`statplan` A data.table containing FCIP rate elements, including at least:

commodity_year Year of the rate observation.

state_code, county_code County identifiers.

commodity_code Crop identifier.

insured_area, lcr, contiguous_state_code, contiguous_county_code Fields required by `estimate_fcip_unloaded_rate()`.

Details

1. **Task list**: Identify all unique (state, county) pairs with data in the 2-21 years before `year`.
2. **Unloaded-rate calculation**: For each county in `task_list`, call `estimate_fcip_unloaded_rate()` on the same 2-21 year window to get `tau`. Errors return `NULL` so processing continues.
3. **Contiguous-county smoothing**:
   - Build a lookup table of contiguous counties (using `contiguous_county`).
   - For each contiguous group, compute the mean `tau` to get `tau_c`.
4. **Merge & fill**: Left-join the raw `adm` and `contiguous_adm`; replace any zero/NA/Inf `tau` with the group mean `tau_c` into `tau_sob`.
5. **Cleanup**: Drop helper columns (`tau`, `tau_c`), remove invalid rows, add `commodity_year`, and return the result.

Value

A data.table with one row per county-crop for the specified `year`, containing:

state_code, county_code, commodity_code Keys.

tau_sob Smoothed unloaded rate (uses contiguous-county means to fill zeros/NAs).

commodity_year The input `year`, repeated.

See Also

`estimate_fcip_unloaded_rate`

---

estimate_fcip_unloaded_rate

Estimate FCIP Unloaded (County) Rates

Description

Computes the unloaded loss cost rates (`tau`) for counties based on the FCIC Rate Methodology Handbook (2009), pp. 65-70.

Usage

```
estimate_fcip_unloaded_rate(
  statplan,
  year = 2011,
  crop = NULL,
  state = NULL,
  county = NULL
)
```

Arguments

`statplan`	A data.table containing FCIP rate elements with at least the columns: **state_code, county_code** Identifiers for each county. **contiguous_state_code, contiguous_county_code** Mapping to county group. **insured_area** Total insured acres in the county. **lcr** Loss Cost Rate for each county. **commodity_code** Crop identifier.
`year`	Integer. Crop year for which rates are being estimated (currently not used but reserved for future subsetting).
`crop`	Optional vector of commodity codes to filter by crop.
`state`	Optional vector of state codes to restrict the analysis.
`county`	Optional vector of county codes to restrict the analysis.

Details

1. **Target data** is filtered to the selected state(s)/county(ies).
2. **Group data** finds contiguous-county groupings, unions them with the target.
3. Computes group-level statistics:

- `c_alpha`: mean insured acres
- `c_u`: mean LCR
- `c_a`: variance of LCR

1. Computes target county statistics:

- `c_x`: mean LCR
- `c_v`: variance of LCR
- `c_net_acre`: total insured acres

1. Applies the blending formula

$$\tau = Z\,x + (1 - Z)\,u, \quad Z = P/(P + K)$$

where $P = c_{net_acre}/c_\alpha, \quad K = c_v/c_a$.

Value

A `data.frame` with columns:

state_code, county_code, commodity_code Keys identifying county and crop.

tau Estimated FCIP county unloaded rate.

References

FCIC Rate Methodology Handbook APH (2009), pp. 65-70. https://legacy.rma.usda.gov/pubs/2008/ratemethodology.pdf

---

FCIP_FORCE_CHARACTER_KEYS

Column names to coerce to character

Description

A character vector of column names that should be converted to character during data ingestion and cleaning.

Usage

```
FCIP_FORCE_CHARACTER_KEYS
```

Format

A `character` vector of column names.

Value

A `character` vector of field names to coerce to character

FCIP_FORCE_NUMERIC_KEYS

Column names to coerce to numeric

Description

A character vector of column names that should be converted from character to numeric during data ingestion and cleaning.

Usage

```
FCIP_FORCE_NUMERIC_KEYS
```

Format

A `character` vector of column names.

Details

The following fields, although often stored as text, represent numeric values and must be coerced for accurate calculation and analysis:

- `commodity_year`: Crop year of the record.
- Fields in `FCIP_INSURANCE_POOL`: `state_code`, `county_code`, `commodity_code`, `type_code`, `practice_code`.
- `record_category_code`: Category of the record.
- `insurance_plan_code`: Code for the insurance plan.
- `coverage_level_percent`: Coverage level percentage.

Value

A `character` vector of field names to coerce to numeric.

Examples

```
## Not run:
# View default keys
FCIP_FORCE_NUMERIC_KEYS

# Extend with a custom numeric field
rFarmPolicySim:::FCIP_FORCE_NUMERIC_KEYS <- c(
  rFarmPolicySim:::FCIP_FORCE_NUMERIC_KEYS,
  "custom_numeric_field"
)

## End(Not run)
```

FCIP_INSURANCE_ELECTION

Insurance election identifier fields

Description

A character vector of column names that define an insurance election within the Federal Crop Insurance Program (FCIP). Each field corresponds to an attribute of a policy election.

Usage

```
FCIP_INSURANCE_ELECTION
```

Format

A `character` vector of field names:

unit_structure_code Structure of the insured unit (e.g., basic, optional).

insurance_plan_code Code for the insurance product (e.g., MPCI, CRC).

coverage_type_code Type of coverage (e.g., Actual/Assumed Yield, Yield Protection).

coverage_level_percent Elected coverage level as a percentage of approved yield or price.

Value

A `character` vector of field names used to specify insurance elections.

Examples

```
## Not run:
# Default election fields
FCIP_INSURANCE_ELECTION

# Override to include only plan and coverage level
rFarmPolicySim:::FCIP_INSURANCE_ELECTION <- c(
```

```
  "insurance_plan_code",
  "coverage_level_percent"
)

## End(Not run)
```

FCIP_INSURANCE_ELECTION_RCODED

Insurance election identifier fields (recoded)

Description

A character vector of recoded column names that specify an insurance election within the Federal Crop Insurance Program (FCIP). These fields correspond to recoded versions of the original election attributes.

Usage

```
FCIP_INSURANCE_ELECTION_RCODED
```

Format

A `character` vector of field names:

unit_structure_recode Recoded unit structure (e.g., basic, optional).

insurance_plan_recode Recoded insurance plan code (e.g., APH, YP).

coverage_type_code Coverage type code (unchanged).

coverage_level_percent unchanged

Value

A `character` vector of recoded insurance election field names.

Examples

```
## Not run:
# View the default recoded election fields
FCIP_INSURANCE_ELECTION_RCODED

# Override to drop the recoded plan field
rFarmPolicySim::FCIP_INSURANCE_ELECTION_RCODED <- c(
  "unit_structure_recode",
  "coverage_type_code",
  "coverage_level_percent"
)

## End(Not run)
```

FCIP_INSURANCE_POOL *Insurance pool identifier fields*

Description

A character vector of column names that together define a unique insurance pool in the Federal Crop Insurance Program (FCIP).

Usage

```
FCIP_INSURANCE_POOL
```

Format

A `character` vector of field names.

Details

Insurance pools represent the most granular level of rate making within FCIP. Each pool is uniquely identified by the combination of:

- **state_code**: State FIPS code
- **county_code**: County FIPS code
- **commodity_code**: Crop commodity code
- **type_code**: Crop type (e.g., grain vs. silage)
- **practice_code**: Production practice (e.g., irrigated, organic)

Value

A `character` vector specifying the columns used to define each FCIP insurance pool.

Examples

```
## Not run:
# Default insurance pool fields
FCIP_INSURANCE_POOL

# Override to a subset of the original fields
rFarmPolicySim:::FCIP_INSURANCE_POOL <- c(
  "state_code", "county_code", "commodity_code"
)

## End(Not run)
```

fixed_effect_model_data_prep

Prepare and demean data for fixed-effects models

Description

This function

1. Filters to complete cases on the specified panel, time, weight, variables, and output
2. If `output` is NULL, creates a dummy output column filled with 1s
3. Drops any panel with only one observation
4. Computes within-panel means for the output + each variable in `varlist` (`_mean_i`)
5. Computes overall sample means for the same set of variables (`_mean`)
6. Replaces each variable in `varlist` by `value - within_panel_mean + overall_mean`

Usage

```
fixed_effect_model_data_prep(
  data,
  varlist,
  panel,
  time,
  wvar = NULL,
  output = NULL
)
```

Arguments

`data`	A data.frame or data.table containing the data.
`varlist`	Character vector of variable names to be demeaned.
`panel`	Character vector of column name(s) defining the panel identifier.
`time`	Character scalar name of the time variable.
`wvar`	Character scalar name of a variable to keep but *not* demean (optional, default NULL).
`output`	Character scalar name of an output variable whose means are computed but not altered; if NULL, a dummy column named `"output"` is created (optional, default NULL).

Value

A list with components

- **data**: a data.table containing
  - the original `panel`, `time`, `wvar`, `varlist`, and `output` columns
  - two mean columns for each of `c(output, varlist)`: `<name>_mean_i` (within-panel) and `<name>_mean` (overall)
- **NFE**: the number of panels with more than one observation

generate_toy_farmer_data

Generate a sample farmer-level toy dataset for testing

Description

Generate a sample farmer-level toy dataset for testing

Usage

```
generate_toy_farmer_data(
  n = 100,
  year = 2020,
  state = 18,
  counties = 7:23,
  commodity = 41,
  types = c(26, 16),
  practices = c(2, 3, 997),
  unit_codes = c("BU", "OU", "EU"),
  plan_codes = c(1:6, 16:17, 90),
  coverage_codes = c("A", "C"),
  cov_level_range = c(0.5, 0.95),
  yield_mean = 100,
  yield_sd = 10,
  price_mean = 2.5,
  price_sd = 1.5,
  price_share = NULL,
  area_share = NULL,
  planted_area = NULL,
  production_history_length = 0:10
)
```

Arguments

`n`	Integer number of farmer records to generate. Default 100.
`year`	Numeric value of commodity year. Default 2020.
`state`	Numeric vector of state FIPS codes. Default 19.
`counties`	Numeric vector of county FIPS codes. Default 127.
`commodity`	Numeric vector of commodity codes. Default 41.
`types`	Numeric vector of type codes. Default 16.
`practices`	Numeric vector of practice codes. Default c(2, 3, 997).
`unit_codes`	Character vector of unit structure codes. Default c("BU", "OU", "EU").
`plan_codes`	Numeric vector of insurance plan codes. Default c(1:6, 16:17, 90).
`coverage_codes`	Character vector of coverage type codes. Default c("A", "C").
`cov_level_range`	Numeric vector of length 2 giving the min and max for coverage_level_percent. Default c(0.8, 0.8).
`yield_mean`	Numeric mean for approved_yield and rate_yield. Default 100.

yield_sd	Numeric standard deviation for approved_yield and rate_yield. Default 10.
price_mean	Numeric mean for expected price. Default 2.5.
price_sd	Numeric standard deviation for expected price. Default 1.5.
price_share	Numeric
area_share	Numeric
planted_area	Numeric
production_history_length	Integer vector of indices for actual_farm_yield columns. Default 0:10.

Value

A data.table with one row per farmer and columns:

- farmid
- commodity_year
- state_code, county_code
- commodity_code, type_code, practice_code
- unit_structure_code, insurance_plan_code
- coverage_type_code, coverage_level_percent
- approved_yield, rate_yield
- reported_acres, insured_share_percent
- price_election_percent, dmage_area_rate, market_year_price
- actual_farm_yield_0 to actual_farm_yield_<max(production_history_length)>

Examples

```
farmdata <- generate_toy_farmer_data()
farmdata <- generate_toy_farmer_data(
  n = 20,
  cov_level_range = c(0.7, 0.9),
  production_history_length = 0:2
)
head(farmdata)
```

---

get_data_release — *Download and cache a data release RDS file from the US-FarmSafetyNet-Lab GitHub repository*

Description

This function constructs the filename <name>_<year>.rds, determines the appropriate GitHub release tag based on the prefix of name ("sob" or "col"), and downloads the file via the piggyback package if it is not already present in the user cache directory. The file is then read into R and returned. Currently supports only Summary of Business releases ("sobscc", "sobcov", "sobtpu") and Cause of Loss releases ("colsom").

Usage

```
get_data_release(name = "sobscc", year = "all")
```

Arguments

name	Character. Base name of the data release (one of "sobscc", "sobcov", "sobtpu", or "colsom"). Defaults to "sobscc".
year	Character or numeric. Year of the data release (e.g., 2020) or "all" for combined releases across all years. Defaults to "all". Availability: * sobscc: "all" only * sobcov: "all" and individual years from 1989-2025 * sobtpu: "all" and individual years from 1999-2025 * colsom: "all" and individual years from 1989-2025

Details

- `tools::R_user_dir("USFarmSafetyNetLab", which = "cache")` locates or creates a per-user cache directory.
- `piggyback::pb_download()` fetches the file from the GitHub repo "ftsiboe/US-FarmSafetyNet-Lab" under the appropriate tag.
- Subsequent calls with the same name and year retrieve the cached file, avoiding repeated downloads.

Value

The R object loaded from the requested RDS file.

Examples

```
## Not run:
  # Download or retrieve cached "sobscc_2021.rds"
  df <- get_data_release("sobscc", 2021)

## End(Not run)
```

---

get_file_info — *Get File Information from a Directory*

Description

Scans a specified directory for files with a given suffix and returns a data frame containing their file paths, sizes in bytes, and sizes in megabytes.

Usage

```
get_file_info(directory = "./data-raw", file_suffix = ".rds")
```

Arguments

directory	A character string specifying the path to the directory to scan. Defaults to `"./data-raw"`.
file_suffix	A character string specifying the file suffix to match. Defaults to `".rds"`.

Value

A data frame with columns:

file_path	Full file path
size_bytes	File size in bytes
size_mb	File size in megabytes

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run:
get_file_info()
get_file_info(directory = "./my-data", file_suffix = ".csv")

## End(Not run)
```

---

get_ice_data — *Download and clean Insurance Control Elements tables*

Description

`get_ice_data()` retrieves all "YTD" ICE (Insurance Control Elements) text files from the specified directory on the RMA public FTP site for one or more years, downloads them to a temporary location, reads them as pipe-delimited data, applies internal cleaning routines, and returns the combined dataset. Original text files are discarded after reading.

Usage

```
get_ice_data(
  years = 2012,
  ice_url =
   "https://pubfs-rma.fpac.usda.gov/pub/References/insurance_control_elements/PASS/",
  selected_ice = NULL
)
```

Arguments

years	Integer vector of calendar years to download (e.g. `2012:2020`). Defaults to `2012`.
ice_url	Character string giving the base URL of the ICE directory on the RMA FTP site. Must end with a slash. Defaults to "`https://pubfs-rma.fpac.usda.gov/pub/References/insur`
selected_ice	Character vector of keyword(s) or regular expressions to filter the filenames. Only ICE files whose names match at least one element of `selected_ice` will be downloaded. If `NULL`, all "YTD" files are processed.

Value

A single `data.table` (invisibly coercible to `data.frame`) containing the cleaned ICE data for all requested years. If no matching files are found or all downloads fail, returns an empty `data.table`.

Examples

```
## Not run:
# Download & process ICE tables for 2018 and 2019,
# filtering for any file with "IceAOExpenseSubsidy" in its name
ice_df <- get_ice_data(
  years        = 2018:2019,
  selected_ice = "IceAOExpenseSubsidy"
)

## End(Not run)
```

---

get_marketing_year_avg_price

Get Marketing Year Average Price for a Single Crop from USDA NASS Quick Stats

---

Description

`get_marketing_year_avg_price()` fetches USDA NASS Quick Stats data for a specified crop (`short_desc`) at one or more aggregation levels (`agg_level_desc`), computes the mean price for the marketing year, joins to official RMA commodity codes, applies necessary unit conversions, and returns a tidy table of marketing-year average prices.

Usage

```
get_marketing_year_avg_price(
  dir_source = "./data-raw/fastscratch/nass/",
  agg_level_desc = c("NATIONAL", "STATE", "COUNTY"),
  short_desc = "CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU"
)
```

Arguments

`dir_source` `character(1)` Path to the directory where Quick Stats files are stored. Defaults to `"./data-raw/fastscratch/nass/"`.

`agg_level_desc` `character` One or more values for the `agg_level_desc` field in the Quick Stats data. Can be `"STATE"` or/and `"NATIONAL"`. Defaults to `"NATIONAL"`.

`short_desc` `character` One or more Quick Stats `short_desc` strings identifying the crop-price series to retrieve. Defaults to `"CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU"`. **Currently, only the following set is supported:** c("OATS - PRICE RECEIVED, MEASURED IN $ / BU", "RYE - PRICE RECEIVED, MEASURED IN $ / BU", "TOBACCO - PRICE RECEIVED, MEASURED IN $ / LB", "CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU", "FLAXSEED - PRICE RECEIVED, MEASURED IN $ / BU", "BARLEY - PRICE RECEIVED, MEASURED IN $ / BU", "BEANS, DRY EDIBLE, INCL CHICKPEAS - PRICE RECEIVED, MEASURED IN $ / CWT", "HAY - PRICE RECEIVED,

MEASURED IN $ / TON", "WHEAT - PRICE RECEIVED, MEASURED IN $ / BU", "COTTON - PRICE RECEIVED, MEASURED IN $ / LB", "SORGHUM, GRAIN - PRICE RECEIVED, MEASURED IN $ / CWT", "SOYBEANS - PRICE RECEIVED, MEASURED IN $ / BU", "SUGARBEETS - PRICE RECEIVED, MEASURED IN $ / TON", "PEAS, DRY EDIBLE - PRICE RECEIVED, MEASURED IN $ / CWT", "SUNFLOWER - PRICE RECEIVED, MEASURED IN $ / CWT", "RICE - PRICE RECEIVED, MEASURED IN $ / CWT", "PEANUTS - PRICE RECEIVED, MEASURED IN $ / LB", "CANOLA - PRICE RECEIVED, MEASURED IN $ / CWT", "MAPLE SYRUP - PRICE RECEIVED, MEASURED IN $ / GALLON", "RICE, LONG GRAIN - PRICE RECEIVED, MEASURED IN $ / CWT", "MILLET, PROSO - PRICE RECEIVED, MEASURED IN $ / BU", "SUGARCANE - PRICE RECEIVED, MEASURED IN $ / TON", "SAFFLOWER - PRICE RECEIVED, MEASURED IN $ / CWT")

Value

A `data.table` with columns:

- `commodity_year` (`integer`): the marketing year
- `commodity_code` (`integer`): NASS commodity code
- `state_code` (`integer`): state FIPS code (if `agg_level_desc` includes `"STATE"`)
- `marketing_year_avg_price` (`numeric`): average price for the marketing year, in dollars per unit
- `data_source` (`character`): always `"USDA NASS Quick Stats"`

See Also

- `process_nass_dataset()` for the underlying data fetch and filtering
- `get_nass_large_datasets()` for downloading the raw Quick Stats files

Examples

```
## Not run:
# Default: national average for corn
get_marketing_year_avg_price()

# Both state and national for wheat
get_marketing_year_avg_price(
  agg_level_desc = c("STATE", "NATIONAL"),
  short_desc     = "WHEAT - PRICE RECEIVED, MEASURED IN $ / BU"
)

## End(Not run)
```

get_nass_census_data *Prepare USDA NASS Census Data for Release*

Description

Iterates over one or more USDA NASS census years, fetching each via `process_nass_dataset()`, and produces three sets of state-level and county-level summaries for each year:

1. Agricultural land metrics by state,
2. Crop insurance totals by state and county,
3. Broad Farm Registry (BRF) census summaries by state and national level.

Usage

```
get_nass_census_data(
  censuses = c(2022, 2017, 2012, 2007, 2002),
  dir_source = "./data-raw/fastscratch/nass/",
  dir_dest = "data-raw/data_release/nass/"
)
```

Arguments

`censuses`	Integer vector. One or more census years to process (e.g. `c(2022, 2017, 2012, 2007, 2002)`).
`dir_source`	Character. Path to the directory containing raw NASS Quick Stats census datasets (default: `"./data-raw/fastscratch/nass/"`).
`dir_dest`	Character. Directory where the processed RDS files will be saved (default: `"data-raw/data_release/nass/"`).

Details

For each year in `censuses`, the function: First, it calls `process_nass_dataset()` with the `large_dataset = paste0("census", census)` argument to load the raw census data into a `data.table`, then renames `commodity_year` to `census_year`. It then performs three blocks of aggregation:

1. **Agricultural Land by State** Filters for ECONOMICS-sector state-level records on cropland, pasture, woodland, and cropland share, cleans and converts the `value` field to numeric, computes the mean by `(census_year, state_code, short_desc)`, recodes `short_desc` to simple labels (`cropland`, `pasture`, `woodland`, `cropland_pct`), pivots wide with `data.table::dcast()`, coerces `census_year` and `state_code` to numeric, and saves `nass_census_agLand_state_<census>.rds`.
2. **Crop Insurance Summaries** Subsets for both cropland and crop-insurance acreage and operation counts plus several farm-related income receipt categories, strips commas and coerces `value` to numeric, drops invalid entries, converts location codes to numeric, filters to insurance-related domains, sums `value` by all relevant grouping fields, and saves `nass_census_insurance_data_<`
3. **BRF Census Aggregates** Filters for producer counts and acres (owned vs. rented), converts `value` to numeric, removes zeros and missing, first computes group-wise means and then sums across `(census_year, state_code, state_alpha, agg_level_desc, unit_desc, domaincat_desc)`, recodes `domaincat_desc` to `ALL_` or `BRF_` plus unit, pivots wide, reorders and renames columns, and saves `nass_census_brf_<census>.rds`.

Value

Character vector of all `nass_census_*.rds` filenames written to `dir_dest`.

See Also

- `process_nass_dataset` for loading raw Quick Stats data

---

get_nass_historical_track_record_crop

Download and Process USDA NASS Historical Crop Track Records

Description

Scrapes the USDA NASS historical track records web page to identify and download the latest `croptrXX.zip` archive, parses its index to locate CSV tables for area planted and harvested by crop, reads and reshapes each table, cleans and standardizes units and variable names, applies special-case unit conversions, and returns a unified time series data frame of crop-level measures (area planted, area harvested, production, yield, price).

Usage

```
get_nass_historical_track_record_crop(
  url = "https://usda.library.cornell.edu/concern/publications/c534fn92g?locale=en",
   dir_source = "./data-raw/fastscratch/nass/"
)
```

Arguments

`url`	Character. URL of the USDA NASS historical track record page.
`dir_source`	Character. Directory into which to download and extract the ZIP file. Default is `"./data-raw/fastscratch/nass/"`. Must exist or be creatable.

Details

1. **Link discovery & download**
   - Reads all `<a>` hrefs from `url`, filters for links containing `"c534fn92g"`, `"zip"`, and `"croptr"`.
   - Extracts the year from each filename and selects the link with the maximum year.
   - Downloads that single ZIP as `croptr.zip` into `dir_source`.
2. **Index parsing**
   - Opens `crop_index.htm` inside the ZIP, extracts the second HTML table as text.
   - Splits on double line breaks, filters to lines mentioning "Area Planted" or "Area Harvested", and excludes summary or non-crop entries.
   - Splits each line into `Page`, `File`, `Description`, then further into `Description` and a year range (`start`/`end`), and derives the uppercase crop name.
3. **CSV reading & reshaping**
   - For each row of the index:
     - Reads the CSV inside the ZIP once to locate header (`h`), unit (`u`), and data (`d`) rows.

    - Reads it again skipping to the data start, then loops over each data column to build a long table with columns `crop`, `crop_yr`, `variable`, `unit`, and `value`.
  - Combines all tables into one data.frame.
4. **Cleaning & standardization**
  - Converts `crop_yr` and `value` to numeric, drops zeros and invalids.
  - Scales values when unit indicates thousands, strips formatting characters, and normalizes variable names.
  - Consolidates crop subtypes (e.g. grain/silage/grazed) under a single `crop`.
5. **Variable mapping & special conversions**
  - Identifies key measures via lookup lists (`Area Planted`, `Area Harvested`, `Production`, `Yield`, `Price`) and relabels them to `"Tracks_*"`.
  - Applies special-case unit conversions (cents to dollars, bales to lbs, cwt to lbs, tons to lbs, etc.), including crop-specific rules (cotton, canola, dry beans, rice, sugarbeet, sugarcane, sunflower).
6. **Final pivot**
  - Pivots the cleaned long table to wide format so each `"Tracks_*"` measure is its own column, and returns a data.frame with columns: `CROP`, `crop_year`, `Tracks_area_planted`, `Tracks_area_harvested`, `Tracks_production`, `Tracks_yield`, `Tracks_price`.

Value

A data.table in which each row corresponds to a crop-year, with standardized track record measures for area planted, area harvested, production, yield, and price.

`get_nass_production_data` *Retrieve and Aggregate NASS Production Data*

Description

This helper function cleans, and aggregates production and area data from the USDA NASS Quick Stats API (via your `process_nass_dataset()` function), for specified geographic aggregation levels (national, state, county). It returns a `data.table` summarizing mean production and area by the chosen levels.

Usage

```
get_nass_production_data(
  dir_source = "./data-raw/fastscratch/nass/",
  source_desc = "SURVEY",
  agg_level_desc = c("NATIONAL", "STATE", "COUNTY")
)
```

Arguments

`dir_source`	Character. Path to the directory where NASS Quick Stats raw QS files are stored (default: `"./data-raw/fastscratch/nass/"`).
`source_desc`	Character. The `source_desc` filter passed to NASS (e.g. `"SURVEY"`).
`agg_level_desc`	Character vector. Which aggregation levels to include: any combination of `"NATIONAL"`, `"STATE"`, and `"COUNTY"`. Controls which code columns (`state_code`, `county_code`) are added.

Details

This function begins by constructing the set of grouping keys (`agg_level_list`), always including the year, commodity name, aggregation descriptor, statistic category, and unit, and then conditionally adding state and/or county codes if those levels are requested. It then invokes `process_nass_dataset()` to fetch the raw crop data for the specified source, sector, domain, country, frequency, reference period, statistic categories, and aggregation levels. Once the data are loaded, any invalid or missing values are removed and the mean of the remaining values is computed for each unique combination of metadata columns. Four separate summaries are then generated: (1) overall production/utilization/class totals, (2) breakdown by commodity class, (3) breakdown by utilization practice, and (4) breakdown by production practice. These four summaries are merged back together, and any rows with unwanted units (e.g., containing a dollar sign) or total rows in the commodity name are filtered out. Next, area metrics are processed by selecting the first non-missing sum across the four summaries and averaging it, and production metrics are handled similarly after filtering out invalid unit-commodity combinations and converting cotton bale values to pounds. Finally, the area and production results are bound together and summed across the chosen grouping keys to produce the final `data.table`.

Value

A `data.table` with one row per combination of:

- `commodity_year`, `commodity_name`,
- chosen geographic codes (`state_code`, `county_code`),
- plus any other aggregation keys. Columns contain summed mean values for production and area.

See Also

- `downloaded_nass_large_datasets` - for downloading and caching USDA NASS Quick Stats large dataset files
- `process_nass_dataset` - for retrieving raw NASS Quick Stats data

`get_state_rental_rates` *Get State Rental Rates for Cropland*

Description

Get State Rental Rates for Cropland

Usage

```
get_state_rental_rates(dir_source = "./data-raw/fastscratch/nass/")
```

Arguments

`dir_source` `character(1)` Path to the directory where Quick Stats files are stored. Defaults to `"./data-raw/fastscratch/nass/"`.

Approximate per-acre cost of crop production using state-level rental rates retrieved from USDA NASS Quick Stats. This function:

1. Loads and aggregates NASS asset values and cash rents by state and year.
2. Joins the two series, excludes non-contiguous states/territories.
3. Estimates missing rents via a panel regression on log asset values, then corrects for systematic bias.
4. Interpolates any remaining missing values using a 5-nearest-neighbor spatial average, iterated twice.

Details

Internally this function relies on:

- `process_nass_dataset()` to pull NASS Quick Stats.
- `plm::pdata.frame()` for panel data setup.
- `lm()` to fit a state-fixed-effects trend model.
- `spdep` to compute spatial lags (5-nearest neighbors).
- `doBy::summaryBy()` for error-ratio corrections.
- `terra` and `tigris` to obtain state geometries.

Value

A `data.frame` with columns:

`NAME` State name

`state_code` Numeric state FIPS code

`commodity_year` Year of the observation

`rent` Adjusted per-acre rent ($/acre)

Examples

```
## Not run:
# Make sure `process_nass_dataset()` and required packages are loaded
df <- get_state_rental_rates()
head(df)

## End(Not run)
```

`get_yu2018_instrument` *Formulate & Merge National Subsidy Rate Instrument (Yu et al., 2018)*

Description

Downloads the historical Summary of Business RDS and computes national subsidy-rate instruments at specified coverage levels, following Yu et al. (2018).

Usage

```
get_yu2018_instrument(
  dt,
  delivery_systems = c("RBUP", "FBUP"),
  plan_codes = c(1:3, 90, 44, 25, 42),
  coverage_levels = c(65, 75)
)
```

Arguments

`dt` sobcov

`delivery_systems` Character vector. Delivery systems to include; default `c("RBUP","FBUP")`.

`plan_codes` Integer vector. Insurance plan codes to include; default `c(1:3, 90, 44, 25, 42)`.

`coverage_levels` Numeric vector. Percent coverage levels to keep; default `c(65, 75)`.

Value

A data.table with columns: commodity_year, subsidy_rate_65, subsidy_rate_75.

---

harmonize_crop_type_codes

Harmonize and summarize crop type codes from raw SOBTPU data

Description

Download the raw Summary of Business by Type, Practice, and Unit Structure (SOBTPU) data, clean and harmonize the crop type names for a fixed set of commodity codes, determine a single dominant type per county, and save the resulting lookup table of type renames.

Usage

```
harmonize_crop_type_codes()
```

Details

This helper function performs the following steps:

1. Downloads the raw SOBTPU data and reads it in.
2. Filters out rows with missing, zero, infinite, or NaN `liability_amount`, and retains only the specified set of commodity codes (wheat, barley, rice, etc.).
3. Recodes `type_name` by commodity:
   - **Wheat (11):** `"DURUM"`, `"SPRING"`, or `"NO TYPE SPECIFIED"`, dropping `"FORAGE WHEAT FOR SEED"`.
   - **Barley (91):** `"SPRING"` or `"WINTER."`
   - **Rice (18):** `"SHORT"`, `"LONG"`, `"MEDIUM"`, or `"NO TYPE SPECIFIED"`, dropping certain japonica types.
   - **Other crops (41, 75, 94, 17, 81):** `"GRAIN"`, `"SILAGE"` (when `type_code == 26`), or "ALL".
4. Summarizes total `liability_amount` by `(commodity_code, state_code, county_code, type_name)`.
5. Pivots to wide form, with one column per `type_name`, filling missing with zero.
6. Determines the single "dominant" type for each county-i.e. the one with positive liability when all other types are zero.
7. Joins this `type_recode` back onto the full dataset and applies final cleanup:
   - If `type_name` is not `"NO TYPE SPECIFIED"`, it overrides the county-level rename.
   - Fills any remaining blanks or `NA` in `type_recode` with `"NO TYPE SPECIFIED"`.
8. Drops any residual invalid liability rows, selects and deduplicates the key columns, adds a `data_source` column, saves to `./data-raw/type_recodes.rds`, and returns the result.

Value

A `data.frame` with columns:

- `commodity_year`
- `state_code`
- `county_code`
- `commodity_code`
- `type_code`
- `type_recode`
- `data_source` (always "Summary of business by type, practice, and unit structure")

Examples

```
## Not run:
# Build and retrieve the lookup of harmonized crop types
lookup_dt <- harmonize_crop_type_codes()

## End(Not run)
```

---

harmonize_election_codes

Harmonize election codes in a data table of insurance elections

---

Description

This function looks for two columns-`unit_structure_code` and `insurance_plan_code`-and creates recoded versions grouping similar codes into broader categories.

Usage

```
harmonize_election_codes(df)
```

Arguments

`df`	A `data.frame` containing, at minimum, one or both of the columns: • `unit_structure_code` • `insurance_plan_code`

Value

A `data.frame` with the same columns as `df` with new columns:

- `unit_structure_recode` (if `unit_structure_code` was present)
- `insurance_plan_recode` (if `insurance_plan_code` was present)

index_for_price_recived *index_for_price_recived*

Description

A combined dataset for index_for_price_recived

Usage

```
data(index_for_price_recived)
```

Format

A data frame with 35 rows and 3 columns covering 1990-2024.

Source

USDA NASS Quick Stats

locate_download_link *Locate the download link for the actuarial data master*

Description

Locate the download link for the actuarial data master

Usage

```
locate_download_link(
  year = 2012,
  adm_url = "https://pubfs-rma.fpac.usda.gov/pub/References/actuarial_data_master/",
  ice_url =
   "https://pubfs-rma.fpac.usda.gov/pub/References/insurance_control_elements/PASS/",
  data_source = "adm"
)
```

Arguments

year	the year of the actuarial data master to download
adm_url	the url where the ADM FTP site is
ice_url	the url where the ICE (insurance control elements) FTP site is
data_source	either "adm" or "ice". Defaults to "adm".

Value

a list of the data and layout file urls with the time the file was last updated on RMA's server

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run: locate_download_link(year = 2012)
```

---

marketing_year_avg_price *marketing_year_avg_price*

Description

A combined dataset for marketing_year_avg_price

Usage

```
data(marketing_year_avg_price)
```

Format

A data frame with 24396 rows and 5 columns covering 1866-2024.

Source

USDA NASS Quick Stats

---

panel_based_spatial_smoothing_estimator *Panel-based spatial smoothing estimator*

Description

This function

1. Constructs spatially-varying treatment interactions (one variable per spatial unit)
2. Applies within-panel/time fixed-effects demeaning to both outcome and interactions
3. Fits an OLS model by hand (`lm.fit`) to recover one coefficient per spatial unit

Usage

```
panel_based_spatial_smoothing_estimator(
  data,
  output,
  treatment,
  time,
  panel,
  spatialvar
)
```

Arguments

`data`	A `data.table` or `data.frame` containing panel data.
`output`	Name of the outcome variable (character scalar).
`treatment`	Name of the treatment variable whose spatial effects we estimate (character scalar).
`time`	Name of the time variable (character scalar).
`panel`	Name(s) of the panel identifier variable(s) (character vector).
`spatialvar`	Name of the spatial grouping variable (e.g. county FIPS; character scalar).

Details

Internally, we

1. Build `treatment_code = I(spatialvar==code) * treatment` for each spatial unit code.
2. Call `fixed_effect_model_data_prep()` to demean the outcome and all `treatment_code` variables.
3. Assemble the design matrix `X = [output_mean_i, treatment_*]` and response `y`.
4. Solve $\hat{\beta} = (\tilde{X}'\tilde{X})^{-1}\tilde{X}'\tilde{y}$ via `lm.fit`.
5. Return a row per spatial unit with its coefficient.

Value

A `data.table` with columns:

- `estimate`: the estimated spatial-unit coefficient
- `county_fips`: the spatial unit identifier (5-digit FIPS)
- `state_code`, `county_code`: parsed FIPS components

`plot_liability_and_acres` *Plot Liability and Net Reported Acres Faceted by a Grouping Variable*

Description

Produces a two-panel bar chart:

1. **Panel A** - Liability (in U.S.\ dollars) by year and group.
2. **Panel B** - Net reported acres (in acres) by year and group.

The panels share the same x-axis (commodity years) and a single legend that is displayed beneath the figure.

Usage

```
plot_liability_and_acres(
  data,
  grouping_variable,
  grouping_name = NULL,
  time_scale_theme = NULL,
  general_theme = NULL,
  label_liability = "Billion U.S. dollars\n",
  label_net_reported_acres = "Million acres\n",
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B", "#BE5E27", "#FFC425",
    "#FEF389", "#D7E5C8", "#9DD9F7", "#51ABA0", "#0F374B")
)
```

Arguments

`data` A `data.frame` containing at least these columns:

- `commodity_year` - numeric or factor; the x-axis.
- `value` - numeric; the bar heights.
- `variable` - factor with the levels `"(A) Liability in U.S. dollars"` and `"(B) Net reported acres"`.
- The column referenced by `grouping_variable`.

`grouping_variable` A character scalar giving the name of the column in `data` that defines the groups (e.g.\ `"commodity"`, `"state"`).

`grouping_name` Optional character scalar used as the legend title. Defaults to the value of `grouping_variable` if `NULL`.

`time_scale_theme` Optional `ggplot2` scale or theme element that controls the x-axis breaks/labels. If `NULL`, the function applies `scale_x_continuous(breaks = unique(data$commodity_year), labels = unique(data$commodity_year))`.

`general_theme` Optional `ggplot2` theme applied to both panels. If `NULL`, `ers_theme()` is used with additional tweaks for font sizes, legend layout, and facet strips.

`label_liability` Y-axis label for Panel A. Default `"Billion U.S. dollars\\n"`.

`label_net_reported_acres` Y-axis label for Panel B. Default `"Million acres\\n"`.

`palette` A character vector of hex colors used to fill the bars. The default is an 11-color palette aligned with ERS/NDSU branding.

Details

Internally the function:

1. Duplicates the column named by `grouping_variable` into `data$grouping_variable` for convenient aesthetic mapping.
2. Builds two separate `ggplot` bar charts (one per `variable` level), applies user themes and palettes, and hides their legends.
3. Extracts a shared legend and x-axis grob with `gtable_filter()`.
4. Assembles everything with `gridExtra::grid.arrange()`.

Value

A named list with five objects:

`fig` A `gtable` containing the assembled two-panel figure, shared x-axis label, and bottom legend.

`figA` The `ggplot` object for Panel A.

`figB` The `ggplot` object for Panel B.

`ldgnd` The extracted legend as a `gtable`.

`xlabT` The shared x-axis grob as a `gtable`.

Examples

```
## Not run:
# Group by commodity with a custom x-axis theme
plot_liability_and_acres(
  data = data_comm,
  grouping_variable = "commodity",
  grouping_name = "Commodity",
  time_scale_theme = ggplot2::scale_x_continuous(
    breaks = seq(2008, 2024, 2),
    labels = seq(2008, 2024, 2)
  ),
  general_theme = ggplot2::theme_minimal()
)

## End(Not run)
```

---

policytime — *Add U.S. Farm Policy Vertical Lines and Labels to a ggplot*

Description

This function overlays vertical lines and text labels on a ggplot object to mark major U.S. agricultural policy events (e.g., Farm Bills or Acts). The vertical lines are drawn at specific years, and labels are positioned based on values provided in the `pty` vector.

Usage

```
policytime(pty, plot, size)
```

Arguments

`pty` A numeric vector of y-axis positions for label placement. The length of the vector determines which policy annotations are added:

- 1st: 1980 Act
- 2nd: 1994 Act
- 3rd: 1996 Farm Bill
- 4th: 2000 Agricultural Risk Protection Act
- 5th: 2008 Farm Bill
- 6th: 2014 Farm Bill

- 7th: 2018 Farm Bill

`plot` A `ggplot` object to which policy lines and labels will be added.

`size` A numeric value indicating the text size of the policy labels.

Details

The lines and labels are added in brown color with dashed lines (`lty=5`), and labels are rotated vertically. Labels are only drawn if the corresponding index exists in `pty`.

Value

A `ggplot` object with added vertical dashed lines and corresponding text labels for each policy year provided.

Examples

```
## Not run:
base_plot <- ggplot(data, aes(x = commodity_year, y = value)) +
  geom_line()
policy_positions <- c(10, 15, 20, 25, 30, 35, 40)
annotated_plot <- policytime(policy_positions, base_plot, size = 3)
print(annotated_plot)

## End(Not run)
```

`prep_fcip_data` *Prepare FCIP Data for Release*

Description

Downloads, processes, and caches multiple Federal Crop Insurance Program (FCIP) datasets-Summary of Business (SOB), Cause of Loss (COL), and Actuarial Data Master (ADM)- saving them as RDS files in a local cache directory structure.

Usage

```
prep_fcip_data(dir_dest = "data-raw/data_release")
```

Arguments

`dir_dest` Character. Base directory under which to store processed FCIP data. Defaults to `"data-raw/data_release"`. Subdirectories `sob`, `col`, and `adm` will be created within `dir_dest` if they do not already exist.

Details

1. **Determine source paths**
   - On Windows, locates the `farmpolicylab` database under the user's OneDrive to Dropbox folder.
   - On non-Windows, uses `~/Database/USA/USDA/`.

2. **Setup local cache directories**
   - Ensures `dir_dest` exists.
   - Creates subfolders `sob`, `col`, and `adm` if missing.
3. **Summary of Business (SOB)**
   - Downloads yearly CSV ZIPs via `download_rma_web_data_files()` for `sobtpu` and `sobcov`, plus field description PDFs.
   - Unzips and reads the historic 1948-1988 SOB data, coerces selected columns to numeric, and saves as `sobscc_1948_1988.rds`.
4. **Cause of Loss (COL)**
   - Downloads yearly CSV ZIPs for `colsom`, plus field description PDF and Excel code listing.
   - Unzips and reads historic 1948-1988 COL data, coerces selected columns (including `indemnity_amount`) to numeric, computes `loss_ratio`, filters invalid rows, and saves as `col_sob_hist.rds`.
5. **Actuarial Data Master (ADM)**
   - Reads RDS files under `rmaFCIPdata/.../base_rate/`, computes `tau_adm = Rr + Rf`, averages by `(crop_yr, state_cd, county_cd, crop_cd)`, and saves `fcip_demand_instruments_from_ad`
   - Reads ERS price projections RDS, renames price columns, averages by `(state_cd, county_cd, typ_cd, cr` standardizes column names, and saves `fcip_commodity_prices.rds`.

Value

A list of three character vectors, naming the files saved in:

- `dir_dest/sob`
- `dir_dest/col`
- `dir_dest/adm`

`prep_nass_data` *Prepare and Save USDA NASS Data for Release*

Description

Downloads and processes multiple USDA NASS datasets-including census, economics, crops, production, marketing-year average prices, and state rental rates-and saves each as an RDS file in a specified directory.

Usage

```
prep_nass_data(
  dir_dest = "data-raw/data_release/nass/",
  dir_source = "./data-raw/fastscratch/nass/"
)
```

Arguments

`dir_dest`	Character. Target directory for saving processed NASS data files (default: `"data-raw/data_release`
`dir_source`	Character. Directory where raw NASS Quick Stats CSVs are stored or will be downloaded (default: `"./data-raw/fastscratch/nass/"`).

Details

This function first ensures that `dir_dest` exists, creating it if necessary. It then downloads the raw NASS Quick Stats datasets for the specified census years, economics, and crops into `dir_source` via `downloaded_nass_large_datasets()`. Next, it retrieves the annual Index for Price Received from the economics dataset with `process_nass_dataset()`, normalizes it to the current year, and saves the result as `nass_survey_index_for_price_received.rds`. It then extracts production and area data at the national, state, and county levels using `get_nass_production_data()`, tags the data source, and writes `nass_survey_production_data.rds`. For commodity price averages, it calls `get_marketing_year_avg_price()` for a predefined list of items, labels the source, and saves `nass_survey_marketing_year_avg_price.rds`. Finally, it obtains annual state cropland cash rent and asset-value data with `get_state_rental_rates()`, notes the bias-correction and interpolation methods used, and saves `nass_survey_state_rental_rates.rds`. The function returns a character vector of all filenames in `dir_dest`.

Value

Character vector of filenames created in `dir_dest`.

See Also

- `downloaded_nass_large_datasets` - download raw NASS Quick Stats CSVs
- `process_nass_dataset` - fetch and reshape raw NASS data
- `get_nass_production_data` - compute production and area metrics
- `get_marketing_year_avg_price` - calculate marketing-year average prices
- `get_state_rental_rates` - assemble state rental rate data

`process_nass_dataset` *Process a USDA NASS Quick Stats dataset by sector and statistic category*

Description

`process_nass_dataset()` downloads (if needed) and reads one or more NASS Quick Stats large datasets files for a given sector, filters the rows by the chosen statistic category plus any additional Quick Stats API parameters, converts and cleans the `value` column, aggregates it by taking its mean over all remaining grouping columns, and then renames that aggregated column to match the requested statistic.

Usage

```
process_nass_dataset(
  dir_source = "./data-raw/fastscratch/nass/",
  large_dataset,
  statisticcat_desc = NULL,
  nassqs_params = NULL
)
```

Arguments

`dir_source` `character(1)` **Length 1.** Path to the directory where Quick Stats large datasets files are stored (and will be downloaded to via `get_nass_large_datasets()`). Defaults to `"./data-raw/fastscratch/nass/"`.

`large_dataset` `character(1)` The Quick Stats `large_dataset` to load (e.g. `"crops"`). one of: "census2002","census2007","census2012","census2017","census2022", "census2007zipcode","censu "animals_products","crops","demographics","economics","environmental"

`statisticcat_desc` `character(1)` **Length 1.** The Quick Stats `statisticcat_desc` to filter on (e.g. `"PRICE RECEIVED"`). After aggregation, the resulting column of mean values will be renamed to `gsub(" ", "_", statisticcat_desc)`.

`nassqs_params` `list` or `NULL` A named list of additional Quick Stats API parameters to filter by (e.g. `"domain_desc"`, `"agg_level_desc"`, `"year"`, etc.). Names must correspond to valid Quick Stats fields. If `NULL` (the default), only `sector_desc` + `statisticcat_desc` filtering is applied. Use `rnassqs::nassqs_params()` to list all valid parameter names.

Details

The full set of valid Quick Stats API parameter names can be retrieved with:

```
rnassqs::nassqs_params()
```

Value

A `data.table` where:

- All original columns have been lowercased.
- Rows have been filtered by `nassqs_params`.
- A `value` column has been converted to numeric (commas stripped), cleaned of non-finite entries, and then aggregated by mean over the remaining columns.
- That aggregated column is renamed to `gsub(" ", "_", statisticcat_desc)`.
- Numeric code columns `state_code`, `country_code`, `asd_code`, plus `commodity_year` and `commodity_name` have been created.

See Also

- `get_nass_large_datasets()` for downloading the raw Quick Stats files

Examples

```
## Not run:
# National annual average price received for all CROPS in 2020:
dt1 <- process_nass_dataset(
  large_dataset       = "crops",
  statisticcat_desc = "PRICE RECEIVED",
  nassqs_params = list( agg_level_desc = "NATIONAL", year = 2020 ))

# State-level marketing-year average price for soybeans:
dt2 <- process_nass_dataset(
  large_dataset       = "crops",
  statisticcat_desc = "PRICE RECEIVED",
```

```
    nassqs_params     = list(
      agg_level_desc      = "STATE",
      short_desc          = "SOYBEANS - PRICE RECEIVED, MEASURED IN $ / BU",
      reference_period_desc = "MARKETING YEAR",
      freq_desc           = "ANNUAL"
    )
  )

  ## End(Not run)
```

`rma_type_recodes` *rma_type_recodes*

Description

A combined dataset for rma_type_recodes

Usage

```
data(rma_type_recodes)
```

Format

A data frame with 232236 rows and 7 columns covering 1999-2025.

Source

USDA-RMA, Summary of business by type, practice, and unit structure

---

`smooth_county_estimates` *Smooth county-level estimates via spatial contiguity*

Description

This function

1. Loads all U.S. county geometries via **urbnmapr**
2. Joins in a user-supplied data.frame of county estimates (by 5-digit FIPS)
3. (Optionally) converts to a **terra** SpatVector for further terra ops
4. Builds a contiguity neighbor list using **sf::st_touches**
5. Iteratively fills any missing estimates by applying a user-supplied function to neighbor values
6. Classifies the smoothed values into `n_classes` using Jenks natural breaks

Usage

```
smooth_county_estimates(
  data,
  fip_col = "fip",
  estimate_col = "estimate",
  iterations = 5,
  fun = mean,
  n_classes = 10
)
```

Arguments

`data`	A data.frame or tibble containing at least two columns: - a 5-digit FIPS code (character) - a numeric estimate to be smoothed
`fip_col`	Name of the FIPS-code column in `fit_df` (default 'fip').
`estimate_col`	Name of the numeric estimate column in `fit_df` (default 'estimate').
`iterations`	Number of smoothing passes to perform (default 5).
`fun`	A function to aggregate neighbor values; must take a numeric vector and return a single numeric (default mean).
`n_classes`	Number of categories for Jenks natural-breaks classification (default 10).

Value

An **sf** object of all U.S. counties with these new columns:

- estimate: your original values (NA where missing)
- estimate_smooth: the same values after neighborhood-function imputation
- estimate_cat: factor giving the Jenks-break category

`standardize_fcip_column_names` *Standardize FCIP Data Frame Column Names*

Description

Rename a set of common Federal Crop Insurance Program (FCIP) data frame columns to consistent, descriptive names. This ensures uniform naming across different data sources and simplifies downstream processing.

Usage

```
standardize_fcip_column_names(df)
```

Arguments

`df`	A `data.frame` containing FCIP data with short column names such as `crop_yr`, `state_cd`, etc.

Value

The input `df` with Standardized column names:

Examples

```
## Not run:
# Suppose fcip_raw is loaded with original column names
fcip_clean <- standardize_fcip_column_names(fcip_raw)
# Now fcip_clean has standardized, descriptive column names

## End(Not run)
```

`state_rental_rates` *state_rental_rates*

Description

A combined dataset for state_rental_rates

Usage

```
data(state_rental_rates)
```

Format

A data frame with 1736 rows and 5 columns covering 1994-2024.

Source

Output from get_state_rental_rates() function

Index

∗ **datasets**
- census_state_beginning_farmer_and_rancher_data, 6
- contiguous_county, 7
- FCIP_FORCE_CHARACTER_KEYS, 12
- FCIP_FORCE_NUMERIC_KEYS, 12
- FCIP_INSURANCE_ELECTION, 13
- FCIP_INSURANCE_ELECTION_RCODED, 14
- FCIP_INSURANCE_POOL, 15
- index_for_price_recived, 30
- marketing_year_avg_price, 31
- rma_type_recodes, 39
- state_rental_rates, 41

analyze_and_summarize_time_series, 3

calculate_mode, 4
calculate_revenue_moments, 5
census_state_beginning_farmer_and_rancher_data, 6
clean_data, 6
contiguous_county, 7

data.table, *11*
download_rma_web_data_files, 8
downloaded_nass_large_datasets, 7, *26*, *37*

ers_theme, 9
estimate_fcip_instruments, 9
estimate_fcip_unloaded_rate, 10, *10*

FCIP_FORCE_CHARACTER_KEYS, 12
FCIP_FORCE_NUMERIC_KEYS, 12
FCIP_INSURANCE_ELECTION, 13
FCIP_INSURANCE_ELECTION_RCODED, 14
FCIP_INSURANCE_POOL, 15
fixed_effect_model_data_prep, 16

generate_toy_farmer_data, 17
get_data_release, 18
get_file_info, 19
get_ice_data, 20
get_marketing_year_avg_price, 21, *37*
get_nass_census_data, 23
get_nass_historical_track_record_crop, 24
get_nass_production_data, 25, *37*
get_state_rental_rates, 26, *37*
get_yu2018_instrument, 27

harmonize_crop_type_codes, 28
harmonize_election_codes, 29

index_for_price_recived, 30

locate_download_link, 30

marketing_year_avg_price, 31

panel_based_spatial_smoothing_estimator, 31
plot_liability_and_acres, 32
policytime, 34
prep_fcip_data, 35
prep_nass_data, 36
process_nass_dataset, *24*, *26*, 37, *37*

rma_type_recodes, 39

smooth_county_estimates, 39
standardize_fcip_column_names, 40
state_rental_rates, 41